



MINISTÉRIO DA EDUCAÇÃO
SECRETARIA DE EDUCAÇÃO PROFISSIONAL
E TECNOLÓGICA
INSTITUTO FEDERAL DE EDUCAÇÃO,
CIÊNCIA E TECNOLOGIA DE GOIÁS
PRÓ-REITORIA DE PESQUISA E
PÓS-GRADUAÇÃO

PROJETO FINAL: ANÁLISE E MODELAGEM DO SISTEMA MEDITRACK

Disciplina: Análise de Sistemas de Informação

Professor: Otávio Calça Xavier

Alunos: João Marcos, Sophia Lannah, Victor Gabriel

Tema: MediTrack - Sistema de Gerenciamento de Medicamentos

Data: 03/02/2026

CONTEÚDO

1	INTRODUÇÃO	4
1.1	Objetivo do Projeto	4
1.2	Descrição do Sistema	4
2	DEFINIÇÃO DO PROBLEMA E ELICITAÇÃO DE REQUISITOS	5
2.1	Escolha do Sistema	5
2.2	Resumo do Sistema	5
2.3	Levantamento de Requisitos	5
2.3.1	Técnicas de Elicitação Aplicadas	5
2.4	Requisitos Elicitados	7
2.4.1	Requisitos Funcionais	7
2.4.2	Requisitos Não Funcionais	9
3	MODELAGEM DOS CASOS DE USO	11
3.1	Introdução	11
3.2	Casos de Uso	11
3.2.1	CU01 - Cadastrar Medicamento	11
3.2.2	CU02 - Listar Medicamentos	13
3.2.3	CU03 - Visualizar Detalhes do Medicamento	14
3.2.4	CU04 - Editar Medicamento	15
3.2.5	CU05 - Marcar Medicamento como Tomado	16
3.2.6	CU06 - Cadastrar Medicamentos em Lote	17
3.2.7	CU07 - Receber Notificação de Lembrete	19
3.2.8	CU08 - Exportar Histórico em PDF	20
3.3	Diagrama de Casos de Uso	21
3.3.1	Resumo dos Casos de Uso e Relacionamentos	23
4	MODELAGEM ESTRUTURAL E COMPORTAMENTAL	24
4.1	Diagrama de Classes	24
4.1.1	Descrição das Classes do Sistema MediTrack	24
4.1.2	Relacionamentos entre Classes	26
4.2	Diagrama de Atividades	30
4.2.1	Processo: Cadastrar Medicamento e Agendar Notificação	30
4.2.2	Fluxo de Atividades	30

4.3	Diagrama de Sequência	34
4.3.1	Processo: Marcar Medicamento como Tomado	34
4.3.2	Participantes	34
4.3.3	Fluxo de Sequência	34
4.4	Diagrama de Estados	37
4.4.1	Estados do Medicamento no Sistema MediTrack	37
4.4.2	Transições	38
4.5	Aplicação de Conceitos OOP	40
4.5.1	Encapsulamento	40
4.5.2	Herança/Implementação	41
4.5.3	Polimorfismo	41
4.5.4	Abstração	41
4.5.5	Relacionamentos	41
5	GERENCIAMENTO E RASTREABILIDADE DOS REQUISITOS	42
5.1	Rastreabilidade dos Requisitos	42
5.1.1	Matriz de Rastreabilidade	42
5.2	Métricas de Rastreabilidade	43
5.2.1	Cobertura de Rastreabilidade	43
5.2.2	Status dos Requisitos	43
5.3	Conclusão	43
6	CONSIDERAÇÕES FINAIS	44
6.1	Principais Desafios e Aprendizados	44
6.1.1	Desafios Enfrentados	44
6.1.2	Aprendizados	45
7	REFERÊNCIAS	45
8	APÊNDICES	47
8.1	Apêndice A - Transcrição da Entrevista	47
8.2	Apêndice B - Análise do Questionário	49
8.3	Apêndice C - Relatório de Observação	50

1 INTRODUÇÃO

Este relatório apresenta o trabalho final da disciplina de Análise de Sistemas de Informação, consistindo na análise e modelagem completa do sistema **MediTrack**, um aplicativo mobile Android para gerenciamento de medicamentos.

1.1 Objetivo do Projeto

O objetivo deste projeto é aplicar os conhecimentos adquiridos ao longo da disciplina na análise e modelagem de um sistema de aplicabilidade real, seguindo os princípios da Engenharia de Requisitos e Modelagem de Sistemas. O trabalho envolve todas as etapas de análise e modelagem, desde a elicitação de requisitos até a documentação e rastreabilidade completa.

1.2 Descrição do Sistema

O MediTrack é um sistema de gerenciamento de medicamentos desenvolvido para auxiliar pacientes e cuidadores no controle da administração de medicamentos. O sistema oferece funcionalidades essenciais como:

- Cadastro de medicamentos com informações detalhadas (nome, dosagem, frequência, horários)
- Notificações automáticas de lembretes para a tomada dos medicamentos
- Histórico completo de uso dos medicamentos
- Exportação de relatórios em PDF para compartilhamento com profissionais de saúde
- Interface simples e intuitiva, adequada para idosos e pessoas com baixo conhecimento técnico

O sistema foi desenvolvido para dispositivos móveis Android, funcionando completamente offline para garantir privacidade e disponibilidade constante. O contexto de uso abrange desde ambientes domésticos até acompanhamento em consultas médicas.

Este documento consolida todas as etapas do projeto, desde a definição do problema e elicitação de requisitos até a modelagem completa do sistema com diagramas UML e a rastreabilidade dos requisitos.

2 DEFINIÇÃO DO PROBLEMA E ELICITAÇÃO DE REQUISITOS

2.1 Escolha do Sistema

O sistema escolhido é o **MediTrack**, um aplicativo mobile Android desenvolvido em Kotlin para controle e gerenciamento de medicamentos. O sistema foi selecionado por tratar-se de uma necessidade real e relevante na área de saúde, onde a adesão medicamentosa é um desafio constante tanto para pacientes quanto para profissionais de saúde.

2.2 Resumo do Sistema

O MediTrack é um sistema de gerenciamento de medicamentos que tem como objetivo auxiliar usuários a controlar de forma eficiente a administração de seus medicamentos. O sistema permite que pacientes ou cuidadores cadastrem medicamentos com suas respectivas informações (nome, dosagem, frequência e horários), recebam notificações de lembretes para a tomada dos medicamentos, acompanhem o histórico de uso e gerem relatórios em PDF para compartilhamento com profissionais de saúde.

O contexto de uso abrange principalmente pessoas que precisam tomar múltiplos medicamentos diariamente, idosos que podem ter dificuldades de memória, pacientes com doenças crônicas que requerem tratamento contínuo, e cuidadores responsáveis pelo gerenciamento medicamentoso de terceiros. O sistema contribui para melhorar a adesão ao tratamento, reduzir erros de medicação e facilitar o acompanhamento do uso de medicamentos ao longo do tempo.

2.3 Levantamento de Requisitos

2.3.1 Técnicas de Elicitação Aplicadas

Foram aplicadas três técnicas de elicitação de requisitos:

2.1.1. Entrevista Estruturada

Participante: Dr. João Silva, Clínico Geral, CRM-GO 12.345, 15 anos de experiência

Local: Consultório do Dr. João Silva, Clínica Saúde Total, Goiânia-GO

Data: 05/12/2025

Horário: 14h30 às 15h15

Duração: 45 minutos

Formato: Entrevista presencial estruturada

A entrevista foi conduzida com um profissional de saúde experiente, que atende principalmente pacientes com doenças crônicas e idosos. O médico demonstrou grande interesse no projeto e forneceu insights valiosos sobre as necessidades reais dos pacientes.

Principais Pontos Identificados:

- Cerca de 40% dos pacientes têm dificuldades com adesão medicamentosa
- Necessidade de registro detalhado de medicamentos (nome, dosagem, horário)
- Importância crítica de notificações confiáveis para melhorar adesão
- Necessidade de histórico detalhado para acompanhamento médico
- Facilidade de uso é essencial, especialmente para idosos
- Possibilidade de exportação de relatórios em PDF para consultas
- Preocupação com privacidade e segurança dos dados de saúde
- Necessidade de cadastro em lote para pacientes com múltiplos medicamentos
- Importância de testar a usabilidade com idosos desde o início do desenvolvimento

Evidência: Transcrição completa da entrevista disponível no Apêndice A (página 47)

2.1.2. Questionário Online

Público-alvo: Usuários potenciais (pacientes e cuidadores)

Período: 01/12/2025 a 08/12/2025

Respostas coletadas: 52 questionários válidos

Principais Necessidades Identificadas:

- 89% dos respondentes consideram notificações muito importantes
- 76% precisam de histórico detalhado
- 68% desejam exportar relatórios para médicos
- 82% preferem interface simples e intuitiva
- 54% gostariam de cadastrar múltiplos medicamentos rapidamente

2.1.3. Observação de Contexto

Local: Farmácia Comunitária do Centro de Goiânia

Endereço: Rua 7, Centro, Goiânia-GO

Data: 03/12/2025

Horário: 14h às 17h

Duração: 3 horas

Observador: Victor Pereira (Estudante de Sistemas de Informação - IFG)

Observações:

- Muitos pacientes anotam medicamentos em papéis
- Dificuldade em lembrar horários corretos
- Necessidade de orientação sobre interações medicamentosas (não implementado nesta versão)
- Preferência por dispositivos móveis para gerenciamento

2.4 Requisitos Elicitados

2.4.1 Requisitos Funcionais

RF01 - Cadastrar Medicamento

O sistema deve permitir que o usuário cadastre um medicamento informando nome, dosagem, frequência e horários de administração.

Prioridade: Alta

Fonte: Entrevista, Questionário, Observação

RF02 - Listar Medicamentos

O sistema deve exibir uma lista de todos os medicamentos cadastrados pelo usuário, apresentando informações básicas como nome, dosagem e status (Pendente, Tomado, Atrasado).

Prioridade: Alta

Fonte: Entrevista, Questionário

RF03 - Visualizar Detalhes do Medicamento

O sistema deve permitir que o usuário visualize detalhes completos de um medicamento específico, incluindo todas as informações cadastradas e histórico de uso.

Prioridade: Média

Fonte: Questionário

RF04 - Editar Medicamento

O sistema deve permitir que o usuário edite informações de um medicamento já cadastrado.

Prioridade: Alta

Fonte: Entrevista, Questionário

RF05 - Marcar Medicamento como Tomado

O sistema deve permitir que o usuário marque um medicamento como “Tomado” após sua administração, atualizando o status e registrando a data/hora.

Prioridade: Alta

Fonte: Entrevista, Questionário, Observação

RF06 - Cadastro em Lote

O sistema deve permitir que o usuário cadastre múltiplos medicamentos simultaneamente através de uma funcionalidade de cadastro em lote.

Prioridade: Média

Fonte: Questionário

RF07 - Notificações de Lembrete

O sistema deve enviar notificações automáticas ao usuário nos horários programados para a tomada de cada medicamento.

Prioridade: Alta

Fonte: Entrevista, Questionário, Observação

RF08 - Histórico de Uso

O sistema deve manter e exibir um histórico completo de todos os medicamentos, incluindo registros de quando foram tomados, quando foram esquecidos e status atual.

Prioridade: Alta

Fonte: Entrevista, Questionário

RF09 - Exportar Histórico em PDF

O sistema deve permitir que o usuário gere e exporte um relatório em PDF contendo o histórico de uso dos medicamentos, adequado para compartilhamento com profissionais de saúde.

Prioridade: Média

Fonte: Entrevista, Questionário

RF10 - Validar Dados de Cadastro

O sistema deve validar que todos os campos obrigatórios (nome, dosagem, frequência, horário) sejam preenchidos antes de permitir o cadastro de um medicamento.

Prioridade: Alta

Fonte: Entrevista

2.4.2 Requisitos Não Funcionais

RNF01 - Performance

O sistema deve responder a ações do usuário em até 2 segundos para operações comuns (cadastro, listagem, atualização de status).

Prioridade: Alta

Fonte: Questionário

RNF02 - Usabilidade

O sistema deve apresentar interface simples e intuitiva, passível de uso por pessoas com baixo conhecimento técnico e idosos.

Prioridade: Alta

Fonte: Entrevista, Questionário, Observação

RNF03 - Confiabilidade

O sistema deve manter a integridade dos dados mesmo em caso de falhas do dispositivo ou da aplicação, utilizando armazenamento persistente local.

Prioridade: Alta

Fonte: Entrevista

RNF04 - Disponibilidade

O sistema deve funcionar sem necessidade de conexão à internet após a instalação.

Prioridade: Média

Fonte: Questionário

RNF05 - Escalabilidade

O sistema deve suportar o cadastro de até 100 medicamentos simultâneos sem degradação de performance.

Prioridade: Média

Fonte: Questionário

RNF06 - Segurança

O sistema deve armazenar dados localmente no dispositivo, sem transmissão para servidores externos, garantindo privacidade das informações de saúde.

Prioridade: Alta

Fonte: Entrevista

RNF07 - Compatibilidade

O sistema deve ser compatível com dispositivos Android versão 7.0 (API 24) ou superior.

Prioridade: Alta

Fonte: Análise de mercado

RNF08 - Notificações

O sistema deve garantir que as notificações sejam entregues mesmo quando o aplicativo não está em execução, utilizando WorkManager.

Prioridade: Alta

Fonte: Entrevista, Questionário

3 MODELAGEM DOS CASOS DE USO

3.1 Introdução

Este documento apresenta a modelagem dos casos de uso do sistema MediTrack, descrevendo as interações entre os atores e o sistema, bem como os fluxos principais, alternativos e de exceção para cada funcionalidade identificada durante a etapa de elicitação de requisitos.

3.2 Casos de Uso

3.2.1 CU01 - Cadastrar Medicamento

Título: Cadastrar Medicamento

Descrição: Este caso de uso permite que o usuário cadastre um novo medicamento no sistema, informando nome, dosagem, frequência e horários de administração.

Atores: Usuário

Pré-condições:

- O usuário deve estar na tela de lista de medicamentos.
- O aplicativo deve estar instalado e funcionando corretamente.

Fluxo Principal:

1. O usuário acessa a tela de lista de medicamentos.
2. O usuário clica no botão “Adicionar Medicamento”.
3. O sistema exibe o formulário de cadastro de medicamento.
4. O usuário preenche o campo “Nome do Medicamento”.
5. O usuário preenche o campo “Dosagem”.
6. O usuário preenche o campo “Frequência”.
7. O usuário preenche o campo “Horário” (ex: 8h, 14h, 20h).
8. O usuário clica no botão “Salvar”.
9. O sistema valida se todos os campos obrigatórios foram preenchidos.
10. O sistema salva o medicamento no banco de dados.

11. O sistema agenda as notificações conforme o horário informado.
12. O sistema retorna à tela de lista de medicamentos.
13. O caso de uso é encerrado.

Fluxos Alternativos:

FA1 - Validação Falha:

- No passo 9, se algum campo obrigatório não foi preenchido:
 - O sistema exibe mensagem de erro solicitando o preenchimento de todos os campos.
 - O fluxo retorna ao passo 4.

FA2 - Cancelar Cadastro:

- A qualquer momento entre os passos 4 e 7, o usuário pode clicar no botão “Voltar”:
 - O sistema descarta as informações preenchidas.
 - O sistema retorna à tela de lista de medicamentos.
 - O caso de uso é encerrado.

Fluxos de Exceção:

FE1 - Erro ao Salvar:

- No passo 10, se ocorrer erro ao salvar no banco de dados:
 - O sistema exibe mensagem de erro informando que não foi possível salvar.
 - O sistema mantém o formulário preenchido.
 - O fluxo retorna ao passo 4.

FE2 - Erro ao Agendar Notificação:

- No passo 11, se ocorrer erro ao agendar a notificação:
 - O sistema salva o medicamento normalmente.
 - O sistema registra o erro internamente.
 - O sistema exibe mensagem informando que o medicamento foi salvo, mas a notificação não foi configurada.
 - O fluxo continua no passo 12.

3.2.2 CU02 - Listar Medicamentos

Título: Listar Medicamentos

Descrição: Este caso de uso permite que o usuário visualize uma lista de todos os medicamentos cadastrados, exibindo informações básicas e o status de cada um.

Atores: Usuário

Pré-condições:

- O aplicativo deve estar instalado e funcionando corretamente.
- O usuário deve ter pelo menos um medicamento cadastrado (opcional).

Fluxo Principal:

1. O usuário abre o aplicativo.
2. O sistema carrega os medicamentos do banco de dados.
3. O sistema exibe a tela de lista de medicamentos.
4. O sistema apresenta cada medicamento com: nome, dosagem e status (Pendente, Tomado, Atrasado).
5. O caso de uso é encerrado.

Fluxos Alternativos:

FA1 - Lista Vazia:

- No passo 3, se não houver medicamentos cadastrados:
 - O sistema exibe mensagem informando que nenhum medicamento foi cadastrado.
 - O sistema sugere ao usuário adicionar um medicamento.
 - O caso de uso é encerrado.

Fluxos de Exceção:

FE1 - Erro ao Carregar Dados:

- No passo 2, se ocorrer erro ao carregar do banco de dados:
 - O sistema exibe mensagem de erro informando que não foi possível carregar os medicamentos.
 - O sistema oferece opção de tentar novamente.
 - Se o usuário escolher tentar novamente, o fluxo retorna ao passo 2.
 - Caso contrário, o caso de uso é encerrado.

3.2.3 CU03 - Visualizar Detalhes do Medicamento

Título: Visualizar Detalhes do Medicamento

Descrição: Este caso de uso permite que o usuário visualize informações detalhadas de um medicamento específico selecionado da lista.

Atores: Usuário

Pré-condições:

- O usuário deve estar na tela de lista de medicamentos.
- Deve existir pelo menos um medicamento cadastrado.

Fluxo Principal:

1. O usuário está na tela de lista de medicamentos.
2. O usuário clica em um medicamento da lista.
3. O sistema carrega os dados completos do medicamento selecionado.
4. O sistema exibe a tela de detalhes com todas as informações: nome, dosagem, frequência, horário e status.
5. O caso de uso é encerrado.

Fluxos Alternativos:

FA1 - Editar Medicamento:

- Após o passo 4, o usuário pode clicar em “Editar”:
 - O sistema abre o formulário de edição preenchido com os dados atuais.
 - O fluxo segue para o caso de uso CU04 (Editar Medicamento).

FA2 - Marcar como Tomado:

- Após o passo 4, se o status não for “Tomado”, o usuário pode clicar em “Marcar como Tomado”:
 - O fluxo segue para o caso de uso CU05 (Marcar Medicamento como Tomado).

Fluxos de Exceção:

FE1 - Medicamento Não Encontrado:

- No passo 3, se o medicamento não for encontrado:

- O sistema exibe mensagem de erro informando que o medicamento não foi encontrado.
- O sistema retorna à tela de lista de medicamentos.
- O caso de uso é encerrado.

3.2.4 CU04 - Editar Medicamento

Título: Editar Medicamento

Descrição: Este caso de uso permite que o usuário edite informações de um medicamento já cadastrado.

Atores: Usuário

Pré-condições:

- O usuário deve estar visualizando os detalhes de um medicamento.
- O medicamento deve existir no sistema.

Fluxo Principal:

1. O usuário está na tela de detalhes do medicamento.
2. O usuário clica em “Editar”.
3. O sistema carrega os dados do medicamento no formulário de edição.
4. O sistema exibe o formulário preenchido com os dados atuais.
5. O usuário modifica os campos desejados (nome, dosagem, frequência ou horário).
6. O usuário clica no botão “Salvar”.
7. O sistema valida se todos os campos obrigatórios foram preenchidos.
8. O sistema atualiza o medicamento no banco de dados.
9. O sistema atualiza as notificações agendadas conforme os novos horários.
10. O sistema retorna à tela de detalhes do medicamento atualizado.
11. O caso de uso é encerrado.

Fluxos Alternativos:

FA1 - Validação Falha:

- No passo 7, se algum campo obrigatório não foi preenchido:
 - O sistema exibe mensagem de erro solicitando o preenchimento de todos os campos.
 - O fluxo retorna ao passo 5.

Fluxos de Exceção:

FE1 - Erro ao Atualizar:

- No passo 8, se ocorrer erro ao atualizar no banco de dados:
 - O sistema exibe mensagem de erro informando que não foi possível atualizar.
 - O sistema mantém o formulário com as alterações.
 - O fluxo retorna ao passo 5.

3.2.5 CU05 - Marcar Medicamento como Tomado

Título: Marcar Medicamento como Tomado

Descrição: Este caso de uso permite que o usuário registre que tomou um medicamento, atualizando seu status para “Tomado” e registrando a data/hora da ação.

Atores: Usuário

Pré-condições:

- O usuário deve estar visualizando os detalhes de um medicamento.
- O medicamento deve ter status diferente de “Tomado”.

Fluxo Principal:

1. O usuário está na tela de detalhes do medicamento.
2. O usuário visualiza que o status do medicamento é “Pendente” ou “Atrasado”.
3. O usuário clica no botão “Marcar como Tomado”.
4. O sistema atualiza o status do medicamento para “Tomado”.
5. O sistema registra a data e hora em que o medicamento foi marcado como tomado.
6. O sistema salva a atualização no banco de dados.
7. O sistema atualiza a exibição mostrando o novo status.

8. O botão “Marcar como Tomado” desaparece da interface.
9. O caso de uso é encerrado.

Fluxos Alternativos:

FA1 - Marcar da Lista:

- O usuário pode marcar como tomado diretamente da lista de medicamentos:
 - O usuário está na tela de lista.
 - O usuário clica em uma ação rápida “Marcar como Tomado” no item da lista.
 - O fluxo segue do passo 4 em diante.

Fluxos de Exceção:

FE1 - Erro ao Salvar:

- No passo 6, se ocorrer erro ao salvar no banco de dados:
 - O sistema exibe mensagem de erro informando que não foi possível atualizar o status.
 - O status permanece inalterado.
 - O fluxo retorna ao passo 2.

3.2.6 CU06 - Cadastrar Medicamentos em Lote

Título: Cadastrar Medicamentos em Lote

Descrição: Este caso de uso permite que o usuário cadastre múltiplos medicamentos simultaneamente através de uma funcionalidade de cadastro em lote.

Atores: Usuário

Pré-condições:

- O usuário deve estar na tela de lista de medicamentos.
- O aplicativo deve estar instalado e funcionando corretamente.

Fluxo Principal:

1. O usuário acessa a tela de lista de medicamentos.
2. O usuário clica no botão “Adicionar Medicamento”.
3. O sistema exibe menu com opções: “Adicionar 1 Medicamento” e “Adicionar Múltiplos”.

4. O usuário seleciona “Adicionar Múltiplos”.
5. O sistema exibe o formulário de cadastro em lote com múltiplos campos.
6. O usuário preenche as informações do primeiro medicamento (nome, dosagem, frequência, horário).
7. O usuário adiciona mais medicamentos através do botão “Adicionar Outro”.
8. O sistema adiciona novos campos para preenchimento.
9. O usuário repete os passos 6 e 7 para cada medicamento adicional.
10. O usuário clica no botão “Salvar Todos”.
11. O sistema valida todos os medicamentos preenchidos.
12. O sistema salva todos os medicamentos válidos no banco de dados.
13. O sistema agenda as notificações para cada medicamento salvo.
14. O sistema retorna à tela de lista de medicamentos.
15. O caso de uso é encerrado.

Fluxos Alternativos:

FA1 - Validação Parcial:

- No passo 11, se alguns medicamentos estão válidos e outros não:
 - O sistema salva apenas os medicamentos válidos.
 - O sistema exibe mensagem informando quantos foram salvos e quantos apresentaram erros.
 - O sistema mantém os campos inválidos para correção.
 - O usuário pode corrigir e tentar salvar novamente ou cancelar.

Fluxos de Exceção:

FE1 - Nenhum Medicamento Válido:

- No passo 11, se nenhum medicamento estiver válido:
 - O sistema exibe mensagem informando que nenhum medicamento pôde ser salvo.
 - O sistema solicita a correção dos dados.
 - O fluxo retorna ao passo 6.

3.2.7 CU07 - Receber Notificação de Lembrete

Título: Receber Notificação de Lembrete

Descrição: Este caso de uso descreve o comportamento do sistema ao enviar notificações automáticas para lembrar o usuário de tomar seus medicamentos nos horários programados.

Atores: Sistema (WorkManager), Usuário

Pré-condições:

- Deve existir pelo menos um medicamento cadastrado com horário definido.
- O sistema de notificações do dispositivo deve estar habilitado.
- O aplicativo deve ter permissão para exibir notificações.

Fluxo Principal:

1. O sistema verifica os horários programados dos medicamentos.
2. O sistema identifica que chegou o horário de um medicamento.
3. O sistema envia uma notificação para o dispositivo do usuário.
4. A notificação exibe: nome do medicamento, dosagem e horário.
5. O usuário visualiza a notificação.
6. O caso de uso é encerrado.

Fluxos Alternativos:

FA1 - Usuário Clica na Notificação:

- No passo 5, se o usuário clicar na notificação:
 - O sistema abre o aplicativo.
 - O sistema exibe a tela de detalhes do medicamento notificado.
 - O fluxo segue para o caso de uso CU03 (Visualizar Detalhes do Medicamento).

Fluxos de Exceção:

FE1 - Notificação Não Entregue:

- No passo 3, se a notificação não puder ser entregue:
 - O sistema registra o erro internamente.

- O sistema tenta reagendar a notificação.
- Se persistir o erro, o sistema marca o medicamento como não notificado no histórico.
- O caso de uso é encerrado.

3.2.8 CU08 - Exportar Histórico em PDF

Título: Exportar Histórico em PDF

Descrição: Este caso de uso permite que o usuário gere e exporte um relatório em PDF contendo o histórico de uso dos medicamentos.

Atores: Usuário

Pré-condições:

- O usuário deve estar na tela de lista de medicamentos.
- Deve existir pelo menos um medicamento cadastrado.
- O dispositivo deve ter permissão de escrita em armazenamento externo.

Fluxo Principal:

1. O usuário está na tela de lista de medicamentos.
2. O usuário clica no ícone de “Exportar Histórico” (ícone de compartilhar).
3. O sistema verifica se possui permissão de escrita.
4. O sistema carrega todos os medicamentos e seus históricos.
5. O sistema gera o arquivo PDF com o histórico.
6. O sistema salva o arquivo PDF no armazenamento do dispositivo.
7. O sistema exibe mensagem informando o local onde o arquivo foi salvo.
8. O caso de uso é encerrado.

Fluxos Alternativos:

FA1 - Solicitar Permissão:

- No passo 3, se o sistema não possuir permissão:
 - O sistema solicita permissão de escrita ao usuário.

- Se o usuário conceder permissão, o fluxo continua no passo 4.
- Se o usuário negar permissão, o fluxo segue para FE1.

Fluxos de Exceção:

FE1 - Permissão Negada:

- No passo FA1, se o usuário negar a permissão:
 - O sistema exibe mensagem informando que não é possível gerar o PDF sem permissão.
 - O caso de uso é encerrado.

FE2 - Erro ao Gerar PDF:

- No passo 5, se ocorrer erro ao gerar o PDF:
 - O sistema exibe mensagem de erro informando que não foi possível gerar o arquivo.
 - O caso de uso é encerrado.

3.3 Diagrama de Casos de Uso

O sistema MediTrack possui 8 casos de uso principais, envolvendo o ator Usuário e o Sistema (WorkManager) para notificações automáticas. O diagrama mostra os relacionamentos entre os casos de uso, incluindo dependências (include) e extensões (extend).

```
@startuml Diagrama_Casos_Uso_MediTrack
!theme plain
skinparam backgroundColor #FFFFFF
skinparam handwritten false
skinparam monochrome false
skinparam shadowing true
skinparam roundcorner 10
skinparam maxMessageSize 60
skinparam linetype ortho

skinparam actor {
    BackgroundColor #E1F5FE
    BorderColor #01579B
    FontSize 12
    FontStyle bold
}

skinparam usecase {
    BackgroundColor #F3E5F5
    BorderColor #4A148C
    FontSize 11
    FontStyle bold
}

skinparam rectangle {
    BackgroundColor #FFF9C4
    BorderColor #F57F17
    FontSize 12
    FontStyle bold
}
```

```

}

skinparam note {
    BackgroundColor #FFFDE7
    BorderColor #F9A825
    FontSize 10
}

left to right direction

actor "Usuário" as usuario #LightBlue
actor "Sistema\n(WorkManager)" as sistema #LightGreen

rectangle "MediTrack - Sistema de Gerenciamento de Medicamentos" #FFF9C4 {

    package "Gerenciamento de Medicamentos" {
        usecase "CU01: Cadastrar\nMedicamento" as CU01
        usecase "CU02: Listar\nMedicamentos" as CU02
        usecase "CU03: Visualizar Detalhes\ndo Medicamento" as CU03
        usecase "CU04: Editar\nMedicamento" as CU04
        usecase "CU05: Marcar Medicamento\ncomo Tomado" as CU05
        usecase "CU06: Cadastrar Medicamentos\nem Lote" as CU06
    }

    package "Notificações" {
        usecase "CU07: Receber Notificação\nde Lembrete" as CU07
    }

    package "Relatórios" {
        usecase "CU08: Exportar Histórico\nem PDF" as CU08
    }

    ' Relacionamentos com atores
    usuario --> CU01 : <<uses>>
    usuario --> CU02 : <<uses>>
    usuario --> CU03 : <<uses>>
    usuario --> CU05 : <<uses>>
    usuario --> CU06 : <<uses>>
    usuario --> CU08 : <<uses>>

    sistema --> CU07 : <<triggers>>

    ' Relacionamentos entre casos de uso
    CU03 ..> CU04 : <<include>>
    CU03 ..> CU05 : <<include>>
    CU04 ..> CU01 : <<extend>>
    CU07 ..> usuario : notifica
    CU07 ..> CU03 : <<extend>>

    note right of CU01
        <b>Descrição:</b>
        Permite cadastrar um novo
        medicamento com todas as
        informações necessárias
        (nome, dosagem, frequência, horários)
    end note

    note right of CU06
        <b>Descrição:</b>
        Permite cadastrar múltiplos
        medicamentos de uma vez,
        otimizando o processo de
        cadastro inicial
    end note

    note right of CU07
        <b>Descrição:</b>
        Notificações automáticas
        enviadas nos horários
        programados pelo sistema
        WorkManager do Android
    end note
}

```

Please use 'option handwritten true' to enable handwritten

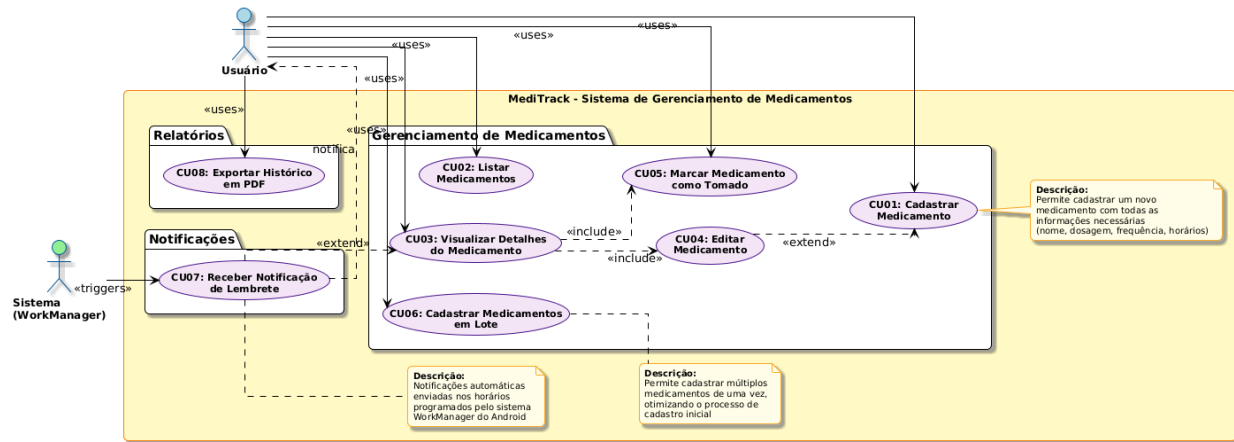


Figura 1: Diagrama de Casos de Uso do Sistema MediTrack

3.3.1 Resumo dos Casos de Uso e Relacionamentos

- **Ator Principal:** Usuário

- **Casos de Uso:**

- CU01: Cadastrar Medicamento
- CU02: Listar Medicamentos
- CU03: Visualizar Detalhes do Medicamento
- CU04: Editar Medicamento
- CU05: Marcar Medicamento como Tomado
- CU06: Cadastrar Medicamentos em Lote
- CU07: Receber Notificação de Lembrete
- CU08: Exportar Histórico em PDF

- **Relacionamentos:**

- CU03 inclui CU04 (para editar)
- CU03 inclui CU05 (para marcar como tomado)
- CU04 estende CU01 (comportamento similar de validação e salvamento)
- CU07 é iniciado pelo Sistema (WorkManager)

4 MODELAGEM ESTRUTURAL E COMPORTAMENTAL

4.1 Diagrama de Classes

4.1.1 Descrição das Classes do Sistema MediTrack

Medication (Entidade)

Classe que representa um medicamento no sistema. Possui informações sobre nome, dosagem, frequência, horários e status.

Atributos:

- **id:** `Int` - Identificador único do medicamento
- **name:** `String` - Nome do medicamento
- **dosage:** `String` - Dosagem do medicamento
- **frequency:** `String` - Frequência de administração
- **schedule:** `String` - Horários de administração (ex: “8h, 14h, 20h”)
- **status:** `String` - Status atual (Pendente, Tomado, Atrasado)

Métodos:

- **isValid():** `Boolean` - Valida se todos os campos obrigatórios foram preenchidos

MedicationDao (Interface)

Interface de acesso a dados (DAO) que define operações de persistência para medicamentos.

Métodos:

- **insert(medication: Medication):** `suspend` - Insere um novo medicamento
- **update(medication: Medication):** `suspend` - Atualiza um medicamento existente
- **getAll():** `Flow<List<Medication>>` - Obtém todos os medicamentos
- **getById(id: Int):** `suspend Medication?` - Obtém um medicamento por ID

MedicationRepository (Interface)

Interface do repositório que abstrai o acesso aos dados.

Métodos:

- `insertMedication(medication: Medication): suspend`
- `updateMedication(medication: Medication): suspend`
- `getAllMedications(): Flow<List<Medication>>`
- `getMedicationById(id: Int): suspend Medication?`

MedicationRepositoryImpl (Implementação)

Implementação do repositório que utiliza o DAO para realizar operações.

Relacionamentos:

- Utiliza `MedicationDao` para acesso aos dados

MedicationDatabase

Classe abstrata que representa o banco de dados Room.

Métodos:

- `medicationDao(): MedicationDao` - Retorna a instância do DAO

Relacionamentos:

- Contém `Medication` como entidade

MedicationViewModel

Classe que gerencia a lógica de apresentação seguindo o padrão MVVM.

Atributos:

- `allMedications: Flow<List<Medication>>` - Fluxo reativo de todos os medicamentos

Métodos:

- `insertMedication(medication: Medication)`
- `updateMedication(medication: Medication)`
- `markMedicationAsTaken(medication: Medication)`
- `getMedicationById(id: Int): Flow<Medication?>`

Relacionamentos:

- Utiliza `MedicationRepository` para operações de dados

NotificationScheduler

Classe responsável por agendar notificações de lembretes.

Métodos:

- `scheduleNotification(medication: Medication)` - Agenda notificações para um medicamento

NotificationWorker

Classe que executa o trabalho de envio de notificações usando WorkManager.

Métodos:

- `doWork(): Result` - Método principal executado pelo WorkManager
- `showNotification(medicationName: String, medicationDosage: String): private` - Exibe a notificação

PdfGenerator

Classe responsável pela geração de relatórios em PDF.

Métodos:

- `generateMedicationHistoryPdf(medications: List<Medication>): File?` - Gera PDF com histórico

AppContainer (Interface)

Interface que define o container de dependências.

Atributos:

- `medicationDatabase: MedicationDatabase`
- `medicationRepository: MedicationRepository`

AppDataContainer (Implementação)

Implementação do container de dependências usando injeção manual.

4.1.2 Relacionamentos entre Classes

O diagrama de classes do sistema MediTrack apresenta os seguintes relacionamentos:

- `MedicationRepositoryImpl` implementa a interface `MedicationRepository`
- `MedicationViewModel` utiliza `MedicationRepository` para operações de dados
- `MedicationDatabase` contém `MedicationDao` e armazena entidades `Medication`

- NotificationScheduler depende de Medication para agendar notificações
- PdfGenerator utiliza Medication para gerar relatórios
- AppDataContainer implementa AppContainer e cria as instâncias de MedicationDatabase e MedicationRepositoryImpl

Código PlantUML do Diagrama de Classes:

```
@startuml Diagrama_Classes_MediTrack
!theme plain
skinparam backgroundColor #FFFFFF
skinparam handwritten false
skinparam monochrome false
skinparam shadowing true
skinparam roundcorner 10
skinparam linetype ortho

skinparam class {
    BackgroundColor #E3F2FD
    BorderColor #1976D2
    FontSize 11
    FontStyle bold
    AttributeFontSize 10
    AttributeFontStyle normal
    MethodFontSize 10
    MethodFontStyle italic
}

skinparam interface {
    BackgroundColor #F1F8E9
    BorderColor #558B2F
    FontSize 11
    FontStyle bold
    AttributeFontSize 10
    AttributeFontStyle normal
    MethodFontSize 10
    MethodFontStyle italic
}

skinparam abstract {
    BackgroundColor #FFF3E0
    BorderColor #E65100
    FontSize 11
    FontStyle bold
    AttributeFontSize 10
    AttributeFontStyle normal
    MethodFontSize 10
    MethodFontStyle italic
}

skinparam package {
    BackgroundColor #F5F5F5
    BorderColor #616161
}

skinparam arrow {
    Color #424242
    Thickness 2
}

package "Entidades" {
    class Medication {
        - id: Long
        - name: String
        - dosage: String
        - frequency: String
        - schedule: String
    }
}
```

```

        - status: MedicationStatus
        - createdAt: LocalDateTime
        - updatedAt: LocalDateTime
        --
        + isValid(): Boolean
        + markAsTaken(): void
        + markAsDelayed(): void
    }

    enum MedicationStatus {
        PENDENTE
        TOMADO
        ATRASADO
    }
}

package "Persistência" {
    interface MedicationDao {
        + insert(medication: Medication): Long
        + update(medication: Medication): void
        + delete(id: Long): void
        + getAll(): Flow<List<Medication>>
        + getById(id: Long): Flow<Medication?>
        + getByStatus(status: MedicationStatus): Flow<List<Medication>>
    }

    abstract class MedicationDatabase {
        + medicationDao(): MedicationDao
        {abstract}
    }

    class AppDatabase {
        - INSTANCE: AppDatabase
        + medicationDao(): MedicationDao
        + getInstance(context: Context): AppDatabase
    }
}

package "Repositório" {
    interface MedicationRepository {
        + insertMedication(medication: Medication): Flow<Long>
        + updateMedication(medication: Medication): Flow<Unit>
        + deleteMedication(id: Long): Flow<Unit>
        + getAllMedications(): Flow<List<Medication>>
        + getMedicationById(id: Long): Flow<Medication?>
        + markMedicationAsTaken(id: Long): Flow<Unit>
    }

    class MedicationRepositoryImpl {
        - medicationDao: MedicationDao
        --
        + insertMedication(medication: Medication): Flow<Long>
        + updateMedication(medication: Medication): Flow<Unit>
        + deleteMedication(id: Long): Flow<Unit>
        + getAllMedications(): Flow<List<Medication>>
        + getMedicationById(id: Long): Flow<Medication?>
        + markMedicationAsTaken(id: Long): Flow<Unit>
    }
}

package "Apresentação" {
    class MedicationViewModel {
        - medicationRepository: MedicationRepository
        - _allMedications: MutableStateFlow<List<Medication>>
        - _selectedMedication: MutableStateFlow<Medication?>
        --
        + allMedications: StateFlow<List<Medication>>
        + selectedMedication: StateFlow<Medication?>
        + insertMedication(medication: Medication): Job
        + updateMedication(medication: Medication): Job
        + deleteMedication(id: Long): Job
        + markMedicationAsTaken(id: Long): Job
        + getMedicationById(id: Long): Job
    }
}

```

```

    }
}

package "Notificações" {
    class NotificationScheduler {
        - context: Context
        - workManager: WorkManager
        --
        + scheduleNotification(medication: Medication): void
        + cancelNotification(medicationId: Long): void
        + rescheduleAllNotifications(): void
        - createWorkRequest(medication: Medication, time: LocalTime): PeriodicWorkRequest
    }

    class NotificationWorker {
        - applicationContext: Context
        - workerParams: WorkerParameters
        --
        + doWork(): Result
        - showNotification(medicationName: String, medicationDosage: String): void
        - createNotificationChannel(): void
    }
}

package "Relatórios" {
    class PdfGenerator {
        - context: Context
        - document: Document
        --
        + generateMedicationHistoryPdf(medications: List<Medication>): File?
        + generateMedicationReport(medication: Medication): File?
        - createDocumentHeader(): void
        - createMedicationTable(medications: List<Medication>): void
        - savePdfFile(document: Document): File?
    }
}

package "Injeção de Dependências" {
    interface AppContainer {
        + medicationDatabase: MedicationDatabase
        + medicationRepository: MedicationRepository
    }

    class AppDataContainer {
        - context: Context
        - _medicationDatabase: MedicationDatabase
        - _medicationRepository: MedicationRepository
        --
        + medicationDatabase: MedicationDatabase
        + medicationRepository: MedicationRepository
    }
}

' Relacionamentos - Entidades
Medication --> MedicationStatus : uses

' Relacionamentos - Persistência
AppDatabase --|> MedicationDatabase : extends
MedicationDatabase "1" *-- "0..*" Medication : stores
MedicationDatabase o-- MedicationDao : provides

' Relacionamentos - Repositório
MedicationRepositoryImpl ..|> MedicationRepository : implements
MedicationRepositoryImpl *-- MedicationDao : uses

' Relacionamentos - Apresentação
MedicationViewModel --> MedicationRepository : uses

' Relacionamentos - Notificações
NotificationScheduler --> Medication : uses
NotificationScheduler --> NotificationWorker : creates
NotificationWorker ..> Medication : uses

```

```

' Relacionamentos - Relatórios
PdfGenerator --> Medication : uses

' Relacionamentos - Injeção de Dependências
AppDataContainer ..|> AppContainer : implements
AppDataContainer *-- MedicationDatabase : creates
AppDataContainer *-- MedicationRepositoryImpl : creates

@enduml

```

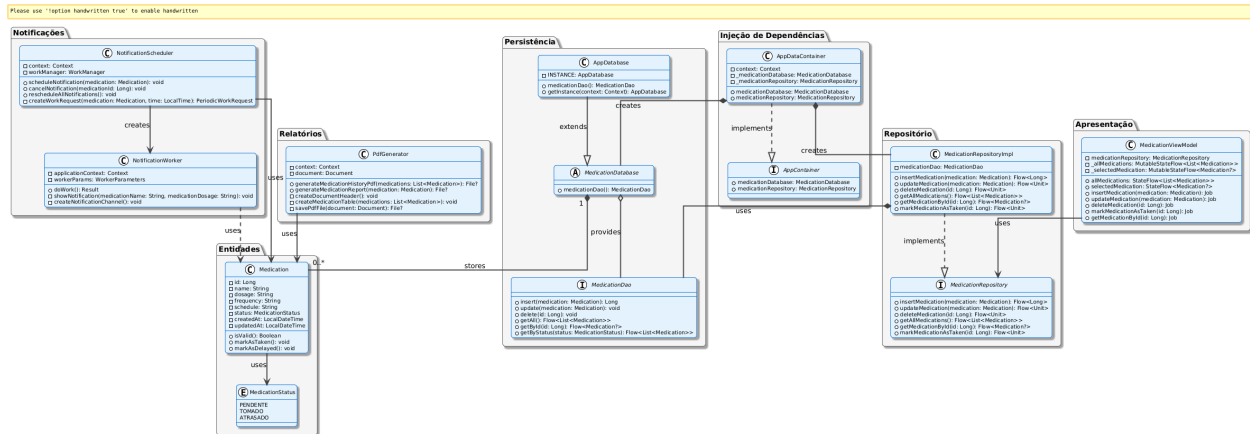


Figura 2: Diagrama de Classes do Sistema MediTrack

4.2 Diagrama de Atividades

4.2.1 Processo: Cadastrar Medicamento e Agendar Notificação

Este processo envolve desde o preenchimento do formulário pelo usuário até o agendamento das notificações pelo sistema, sendo um processo crítico para o funcionamento do aplicativo.

4.2.2 Fluxo de Atividades

1. Usuário acessa formulário de cadastro
2. Sistema exibe campos do formulário
3. Usuário preenche informações (nome, dosagem, frequência, horário)
4. Sistema valida campos obrigatórios
5. Se inválido: exibe mensagem de erro e retorna ao passo 3
6. Se válido: cria objeto Medication

7. Sistema salva medicamento no banco de dados
8. Sistema agenda notificações para os horários informados
9. Sistema parse do horário (ex: “8h, 14h, 20h”)
10. Para cada horário:
 - Sistema calcula delay até o horário
 - Sistema cria PeriodicWorkRequest
 - Sistema agenda notificação com WorkManager
11. Sistema retorna à tela de lista
12. Fim do processo

Código PlantUML do Diagrama de Atividades:

```
@startuml Diagrama_Atividades_Cadastrar_Medicamento
!theme plain
skinparam backgroundColor #FFFFFF
skinparam handwritten false
skinparam monochrome false
skinparam shadowing true
skinparam roundcorner 10
skinparam activity {
    BackgroundColor #E1F5FE
    BorderColor #01579B
    FontSize 11
    FontStyle bold
}

skinparam partition {
    BackgroundColor #F3E5F5
    BorderColor #4A148C
    FontSize 11
    FontStyle bold
}

skinparam note {
    BackgroundColor #FFFDE7
    BorderColor #F9A825
    FontSize 10
}

start

partition "Interface do Usuário" {
    :Usuário acessa tela de\ncadastro de medicamento;
    :Sistema exibe formulário\ncom campos obrigatórios;
    note right
        Campos: Nome, Dosagem,
        Frequência, Horários
    end note
}

partition "Validação" {
    :Usuário preenche informações\ndo medicamento;
    :Sistema valida campos obrigatórios;

    if (Todos os campos válidos?) then (não)
        :Sistema exibe mensagens\nde erro específicas;
    end
}
```

```

        :Usuário corrige informações;
        :Sistema valida campos obrigatórios;
    endif
}

partition "Persistência" {
    :Sistema cria objeto Medication\ncom dados informados;
    :Sistema define status inicial\ncomo PENDENTE;
    :Sistema salva medicamento\ndo banco de dados;

    if (Medicamento salvo com sucesso?) then (não)
        :Sistema exibe mensagem\nde erro ao salvar;
        stop
    endif
}

partition "Agendamento de Notificações" {
    :Sistema faz parse do campo\nschedule (ex: "8h, 14h, 20h");

    :Sistema extrai lista\nde horários;

    while (Há mais horários para processar?) is (sim)
        :Sistema calcula delay\ndo horário atual até o horário\nda primeira dose;

        :Sistema cria PeriodicWorkRequest\ncom frequência diária;

        :Sistema agenda notificação\nusando WorkManager;

        note right
            WorkManager garante
            que a notificação será
            exibida mesmo se o app
            estiver em background
        end note
    endwhile (não há mais horários)

    :Sistema registra todas as\nnotificações agendadas;
}

partition "Feedback ao Usuário" {
    :Sistema atualiza lista\nde medicamentos;
    :Sistema exibe mensagem\nde sucesso;
    :Sistema retorna à tela\nde listagem;
    stop
}

@enduml

```

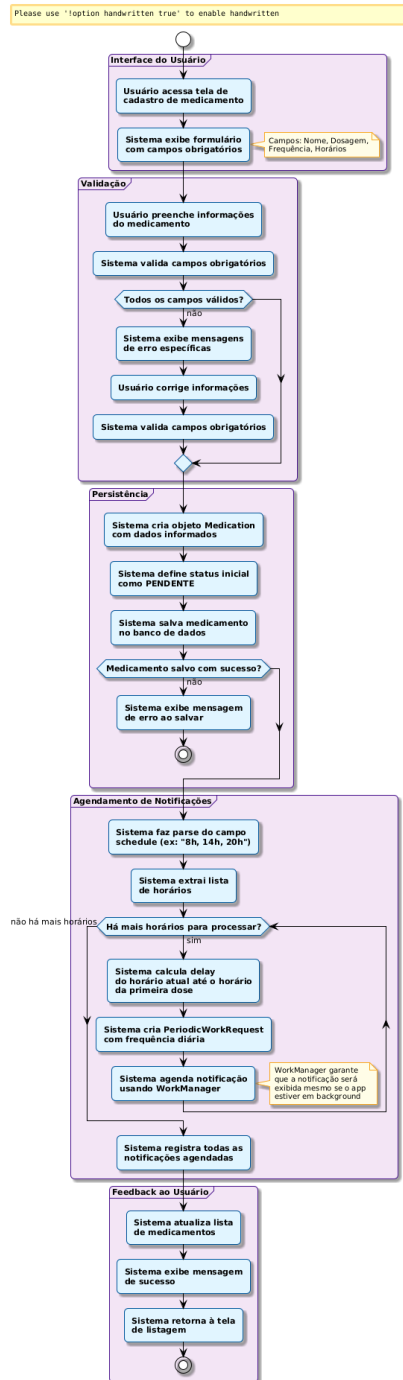



Figura 3: Diagrama de Atividades - Cadastrar Medicamento e Agendar Notificação

4.3 Diagrama de Sequência

4.3.1 Processo: Marcar Medicamento como Tomado

Este processo mostra a sequência de chamadas entre os componentes do sistema quando o usuário marca um medicamento como tomado.

4.3.2 Participantes

- **Usuário:** Ator que interage com o sistema
- **MedicationDetailScreen:** Tela de detalhes do medicamento
- **MedicationViewModel:** ViewModel que gerencia a lógica
- **MedicationRepository:** Camada de repositório
- **MedicationDao:** Camada de acesso a dados
- **MedicationDatabase:** Banco de dados Room

4.3.3 Fluxo de Sequência

1. Usuário clica em “Marcar como Tomado”
2. MedicationDetailScreen chama `markMedicationAsTaken()` no ViewModel
3. MedicationViewModel cria cópia do medicamento com status “Tomado”
4. MedicationViewModel chama `updateMedication()` no Repository
5. MedicationRepository chama `update()` no DAO
6. MedicationDao executa UPDATE no Database
7. MedicationDatabase persiste a atualização
8. O fluxo reativo (**Flow**) notifica os observadores
9. MedicationViewModel atualiza o estado
10. MedicationDetailScreen atualiza a interface
11. Botão “Marcar como Tomado” desaparece da tela

Código PlantUML do Diagrama de Sequência:

```

@startuml Diagrama_Sequencia_Marcar_Como_Tomado
!theme plain
skinparam backgroundColor #FFFFFF
skinparam handwritten false
skinparam monochrome false
skinparam shadowing true
skinparam roundcorner 10
skinparam maxMessageSize 60
skinparam linetype ortho

skinparam actor {
    BackgroundColor #E1F5FE
    BorderColor #01579B
    FontSize 12
    FontStyle bold
}

skinparam participant {
    BackgroundColor #F3E5F5
    BorderColor #4A148C
    FontSize 11
    FontStyle bold
}

skinparam database {
    BackgroundColor #E8F5E9
    BorderColor #2E7D32
    FontSize 11
    FontStyle bold
}

skinparam arrow {
    Color #424242
    Thickness 2
}

skinparam note {
    BackgroundColor #FFFDE7
    BorderColor #F9A825
    FontSize 10
}

actor "Usuário" as Usuario #LightBlue
participant "MedicationDetailScreen" as Screen #LightCoral
participant "MedicationViewModel" as ViewModel #LightGreen
participant "MedicationRepository" as Repository #LightYellow
participant "MedicationDao" as Dao #LightPink
database "Room Database" as Database #LightGreen

Usuario -> Screen: 1. Clica no botão\n"Marcar como Tomado"
activate Screen

Screen -> Screen: 2. Valida se medicamento\nnão está já marcado

alt Medicamento já está marcado
    Screen --> Usuario: Exibe mensagem:\n"Medicamento já foi tomado"
else Medicamento não está marcado
    Screen -> ViewModel: 3. markMedicationAsTaken(\nmedicationId: Long)
    activate ViewModel

    ViewModel -> ViewModel: 4. Cria cópia do medicamento\ncom status = TOMADO

    ViewModel -> ViewModel: 5. Atualiza updatedAt\ncom timestamp atual

    ViewModel -> Repository: 6. updateMedication(\nmedication: Medication)
    activate Repository

    Repository -> Repository: 7. Valida dados\ndo medicamento

    Repository -> Dao: 8. update(medication: Medication)
    activate Dao

```

```

    Dao --> Database: 9. UPDATE medications\nSET status = 'TOMADO',\nupdatedAt = NOW()\nWHERE id = ?
    activate Database

    Database --> Dao: 10. Retorna número de\nlinhas afetadas
    deactivate Database

    alt Atualização bem-sucedida
        Dao --> Repository: 11. Sucesso (rows > 0)
        deactivate Dao

        Repository --> ViewModel: 12. Flow emite\nmedication atualizado
        deactivate Repository

        note right of ViewModel
            O Flow reativo notifica
            automaticamente todos os
            observadores (StateFlow)
            sobre a mudança de estado
        end note

        ViewModel --> ViewModel: 13. Atualiza StateFlow\nallMedications

        ViewModel --> Screen: 14. Estado atualizado\nvia StateFlow
        deactivate ViewModel

        Screen --> Screen: 15. Atualiza UI com\nnovo estado

        Screen --> Screen: 16. Remove botão\n"Marcar como Tomado"

        Screen --> Screen: 17. Exibe indicador\n"Medicamento Tomado"

        Screen --> Usuario: 18. Interface atualizada\ncom feedback visual

    else Erro na atualização
        Dao --> Repository: 11. Erro (rows = 0)
        deactivate Dao

        Repository --> ViewModel: 12. Flow emite erro
        deactivate Repository

        ViewModel --> Screen: 13. Erro ao atualizar
        deactivate ViewModel

        Screen --> Usuario: 14. Exibe mensagem de erro

    end

end

deactivate Screen

@enduml

```

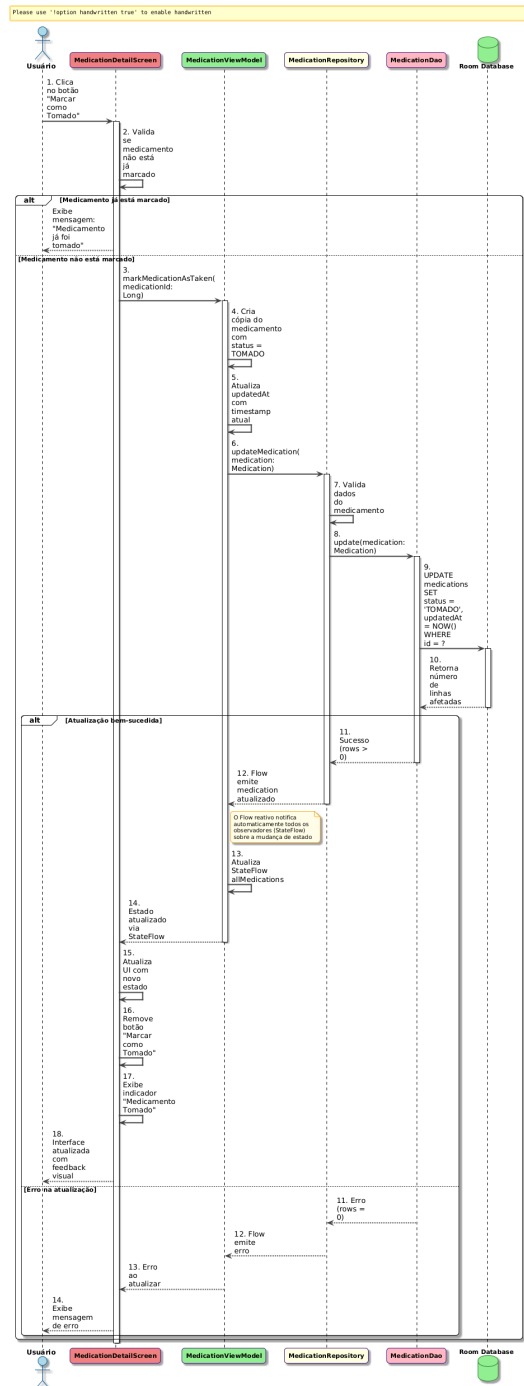


Figura 4: Diagrama de Sequência - Marcar Medicamento como Tomado

4.4 Diagrama de Estados

4.4.1 Estados do Medicamento no Sistema MediTrack

1. **Pendente:** Estado inicial quando o medicamento é cadastrado

2. **Tomado:** Estado quando o usuário marca o medicamento como tomado
3. **Atrasado:** Estado quando o horário passou e o medicamento não foi marcado como tomado (futuro)

4.4.2 Transições

- **Cadastrar:** [Estado inicial] → Pendente
- **Marcar como Tomado:** Pendente → Tomado
- **Marcar como Tomado:** Atrasado → Tomado
- **Horário Passou (sistema):** Pendente → Atrasado (lógica futura)

Código PlantUML do Diagrama de Estados:

```
@startuml Diagrama_Estados_Medicamento
!theme plain
skinparam backgroundColor #FFFFFF
skinparam handwritten false
skinparam monochrome false
skinparam shadowing true
skinparam roundcorner 10

skinparam state {
    BackgroundColor #E3F2FD
    BorderColor #1976D2
    FontSize 11
    FontStyle bold
}

skinparam arrow {
    Color #424242
    Thickness 2
}

skinparam note {
    BackgroundColor #FFFDE7
    BorderColor #F9A825
    FontSize 10
}

[*] --> Pendente : Cadastrar medicamento

state Pendente {
    state "Medicamento cadastrado" as P1
    state "Aguardando horário\nda primeira dose" as P2

    P1 --> P2 : Horário agendado
}

state Atrasado {
    state "Horário da dose passou" as A1
    state "Medicamento não tomado" as A2
    state "Sistema detecta atraso" as A3

    A1 --> A2 : Horário passou
    A2 --> A3 : Sistema verifica status
}

state Tomado {
    state "Usuário marca como tomado" as T1
}
```

```

state "Registrado data/hora" as T2
state "Status finalizado" as T3

T1 --> T2 : Confirmação
T2 --> T3 : Persistido no BD
}

Pendente --> Tomado : marcarComoTomado()\n[usuário confirma]
Pendente --> Atrasado : verificarHorario()\n[horário passou && status = PENDENTE]
Atrasado --> Tomado : marcarComoTomado()\n[usuário confirma]

note right of Pendente
<b>Estado Inicial</b>
Quando o medicamento é
cadastrado, ele inicia
no estado PENDENTE.
O sistema aguarda o
horário programado.
end note

note right of Atrasado
<b>Estado de Atraso</b>
O sistema verifica
periodicamente se o
horário passou sem
o medicamento ser tomado.
Pode ser marcado como
tomado mesmo estando atrasado.
end note

note right of Tomado
<b>Estado Final</b>
Quando o usuário marca
como tomado, o sistema
registra a data e hora
exata da confirmação.
Este é um estado terminal
para aquela dose específica.
end note

@enduml

```

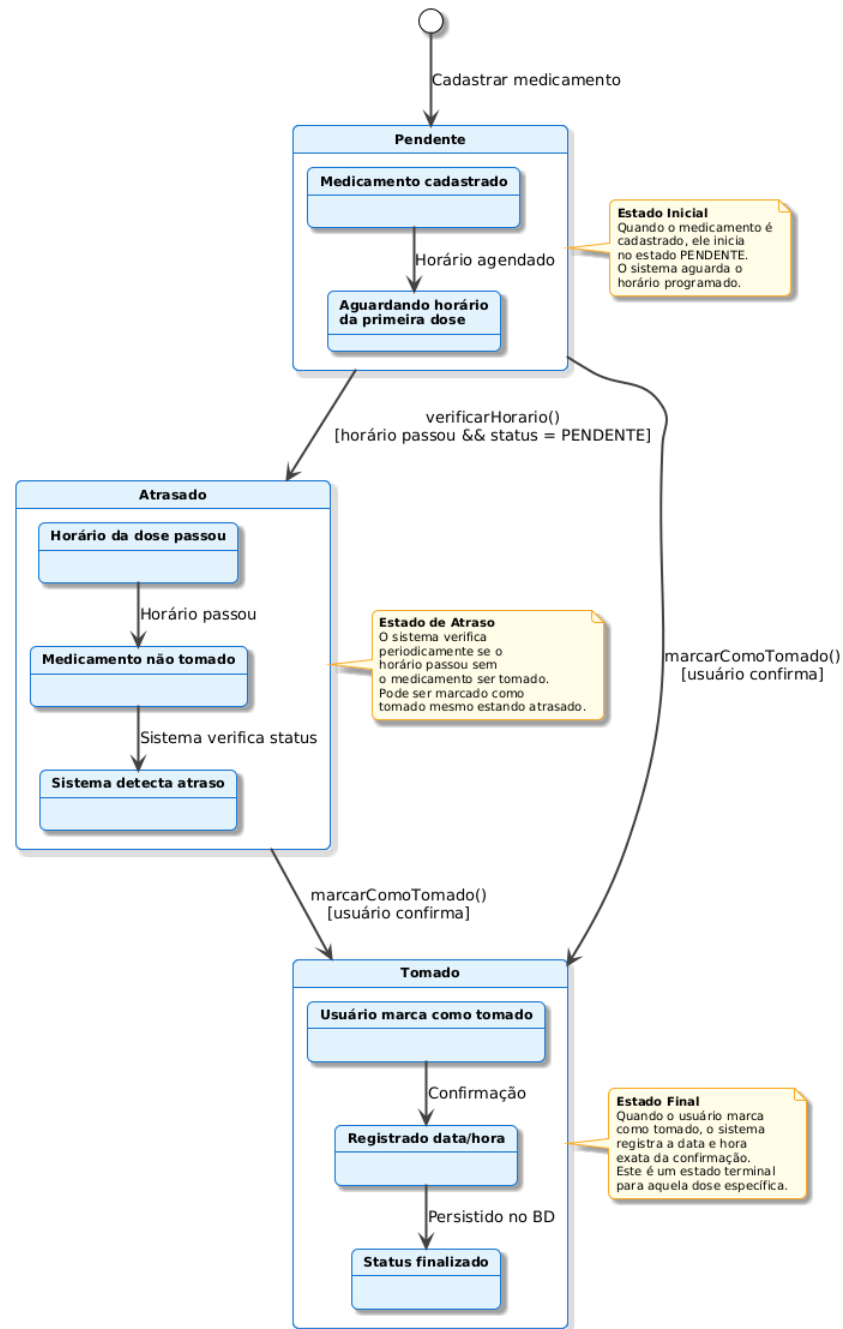


Figura 5: Diagrama de Estados do Medicamento

4.5 Aplicação de Conceitos OOP

4.5.1 Encapsulamento

- Todas as classes encapsulam seus dados e comportamentos

- Atributos privados são acessados através de métodos públicos
- Exemplo: `Medication.isValid()` encapsula a lógica de validação

4.5.2 Herança/Implementação

- `MedicationRepositoryImpl` implementa a interface `MedicationRepository`
- `AppDataContainer` implementa a interface `AppContainer`
- `NotificationWorker` herda de `Worker` (Android `WorkManager`)

4.5.3 Polimorfismo

- O uso de interfaces permite polimorfismo
- `MedicationRepository` pode ser substituído por diferentes implementações
- Diferentes tipos de `Worker` podem ser usados para diferentes tarefas

4.5.4 Abstração

- Interfaces (`MedicationRepository`, `MedicationDao`) abstraem implementações
- A camada de `ViewModel` abstrai a lógica de negócio da UI

4.5.5 Relacionamentos

- **Associação:** `MedicationViewModel` $\rightarrow$ `MedicationRepository`
- **Agregação:** `MedicationDatabase` $\rightarrow$ `MedicationDao`
- **Composição:** `MedicationDatabase` compõe `Medication` (persistência)
- **Dependência:** `NotificationScheduler` depende de `Medication`

5 GERENCIAMENTO E RASTREABILIDADE DOS REQUISITOS

5.1 Rastreabilidade dos Requisitos

5.1.1 Matriz de Rastreabilidade

A rastreabilidade foi estabelecida conectando cada requisito aos seus artefatos relacionados. A seguir está a matriz completa:

Requisitos Funcionais

ID	Requisito	Casos de Uso	Diagramas UML	Implementação
RF01	Cadastrar Medicamento	CU01	Classes, Atividades	AddEditMedicationScreen.kt
RF02	Listar Medicamentos	CU02	Classes	MedicationListScreen.kt
RF03	Visualizar Detalhes	CU03	Classes, Sequência	MedicationDetailScreen.kt
RF04	Editar Medicamento	CU04	Classes, Atividades	AddEditMedicationScreen.kt
RF05	Marcar como Tomado	CU05	Classes, Sequência, Estados	MedicationViewModel.kt
RF06	Cadastro em Lote	CU06	Classes	BulkAddMedicationScreen.kt
RF07	Notificações de Lembrete	CU07	Classes, Atividades	NotificationScheduler.kt NotificationWorker.kt
RF08	Histórico de Uso	CU02, CU03	Classes	MedicationListScreen.kt
RF09	Exportar Histórico PDF	CU08	Classes	PdfGenerator.kt
RF10	Validar Dados	CU01, CU04	Classes	Medication.kt

Requisitos Não Funcionais

ID	Requisito	Artefatos Relacionados	Implementação
RNF01	Performance	Diagrama de Classes	Arquitetura MVVM
RNF02	Usabilidade	Casos de Uso	Interface Jetpack Compose

ID	Requisito	Artefatos Relacionados	Implementação
RNF03	Confiabilidade	Diagrama de Classes	Room Database
RNF04	Disponibilidade	Diagrama de Classes	Armazenamento local
RNF05	Escalabilidade	Diagrama de Classes	Room Database otimizado
RNF06	Segurança	Diagrama de Classes	Armazenamento local
RNF07	Compatibilidade	Documentação	<code>build.gradle.kts</code>
RNF08	Notificações	Diagrama de Atividades	WorkManager

5.2 Métricas de Rastreabilidade

5.2.1 Cobertura de Rastreabilidade

- **Total de Requisitos:** 18 (10 funcionais + 8 não funcionais)
- **Requisitos Rastreados:** 18 (100%)
- **Artefatos Criados:**
 - 8 Casos de Uso
 - 4 Diagramas UML (Classes, Atividades, Sequência, Estados)
 - 1 Diagrama de Casos de Uso

5.2.2 Status dos Requisitos

- **Implementados:** 18/18 (100%)
- **Documentados:** 18/18 (100%)
- **Rastreados:** 18/18 (100%)

5.3 Conclusão

A rastreabilidade foi estabelecida conectando cada requisito aos seus artefatos relacionados através da matriz de rastreabilidade, garantindo que todos os requisitos estão conectados aos seus respectivos casos de uso, diagramas UML e implementações, facilitando a validação e a manutenção do sistema.

6 CONSIDERAÇÕES FINAIS

O projeto MediTrack foi desenvolvido seguindo metodologias de Engenharia de Requisitos e Modelagem de Sistemas, resultando em uma documentação completa e rastreável.

Os principais resultados alcançados foram:

- ✓ Elicitação completa de requisitos utilizando múltiplas técnicas
- ✓ Modelagem detalhada através de 8 casos de uso
- ✓ Criação de 5 diagramas UML (Casos de Uso, Classes, Atividades, Sequência, Estados)
- ✓ Rastreabilidade 100% dos requisitos
- ✓ Documentação completa e organizada

O sistema MediTrack demonstra a aplicação prática dos conhecimentos adquiridos na disciplina, desde a identificação das necessidades dos usuários até a modelagem completa da solução proposta.

6.1 Principais Desafios e Aprendizados

6.1.1 Desafios Enfrentados

Durante o desenvolvimento do projeto, foram enfrentados diversos desafios que contribuíram para o aprendizado:

1. Elicitação de Requisitos:

- **Desafio:** Encontrar participantes adequados para entrevistas e questionários que conhecessem o domínio do sistema de gerenciamento de medicamentos.
- **Solução:** Diversificação de técnicas de elicitação (entrevista com profissional de saúde, questionário online para usuários potenciais, e observação em farmácia) permitiu obter uma visão abrangente das necessidades reais.

2. Modelagem de Casos de Uso:

- **Desafio:** Determinar o nível de detalhamento apropriado para cada caso de uso, equilibrando completude e clareza.
- **Solução:** Seguir o padrão estabelecido na disciplina, incluindo fluxos principais, alternativos e de exceção, garantiu que todos os cenários importantes fossem cobertos.

3. Diagramas UML:

- **Desafio:** Representar corretamente relacionamentos complexos entre classes e modelar processos comportamentais de forma clara e precisa.
- **Solução:** Estudo de padrões UML e revisão de exemplos, além do uso da ferramenta PlantUML, facilitou a criação de diagramas consistentes e bem estruturados.

4. Rastreabilidade:

- **Desafio:** Manter consistência entre requisitos, casos de uso, diagramas e implementação, garantindo que todos os requisitos fossem rastreados adequadamente.
- **Solução:** Criação de uma matriz de rastreabilidade detalhada permitiu estabelecer conexões claras entre todos os artefatos do projeto.

6.1.2 Aprendizados

O desenvolvimento deste projeto proporcionou aprendizados significativos:

- A aplicação de múltiplas técnicas de elicitação (entrevista, questionário e observação) forneceu uma visão mais completa e precisa dos requisitos do sistema.
- Os diagramas UML facilitaram significativamente a compreensão do sistema e ajudaram a identificar inconsistências antes da implementação.
- A rastreabilidade permitiu validar que todos os requisitos foram atendidos e facilita futuras manutenções do sistema.
- A organização em entregas parciais permitiu entregas incrementais e validação contínua.
- O projeto permitiu aplicar na prática conceitos teóricos aprendidos em sala de aula, consolidando o aprendizado.

7 REFERÊNCIAS

Bibliografia:

- SOMMERVILLE, Ian. **Engenharia de Software**. 10ª edição.
- PRESSMAN, Roger S. **Engenharia de Software**. 8ª edição.
- LARMAN, Craig. **Utilizando UML e Padrões**. 3ª edição.

- Documentação oficial Android: <https://developer.android.com>

Ferramentas Utilizadas:

- **Modelagem UML:** PlantUML
- **Documentação:** Markdown, LaTeX
- **Desenvolvimento:** Android Studio, Kotlin, Jetpack Compose

8 APÊNDICES

8.1 Apêndice A - Transcrição da Entrevista

Data: 05/12/2025

Horário: 14h30 às 15h15

Local: Consultório do Dr. João Silva, Clínica Saúde Total, Goiânia-GO

Entrevistador: Victor Pereira (Estudante de Sistemas de Informação - IFG)

Entrevistado: Dr. João Silva, Clínico Geral, CRM-GO 12.345, 15 anos de experiência

Duração: 45 minutos

Formato: Entrevista presencial estruturada

Contexto: A entrevista foi realizada no consultório do médico, em um ambiente reservado. O médico demonstrou interesse no projeto e colaborou ativamente durante toda a entrevista.

Q1: Dr. João, qual sua experiência com pacientes que têm dificuldades para seguir tratamentos medicamentosos?

R: Muito comum, infelizmente. Principalmente idosos e pacientes com múltiplas medicações. O problema maior é esquecer de tomar ou tomar no horário errado. Eu diria que cerca de 40% dos meus pacientes têm alguma dificuldade com adesão medicamentosa. Alguns esquecem completamente, outros tomam em horários diferentes do prescrito, e há aqueles que confundem as dosagens.

Q2: Como você vê a utilidade de um aplicativo de controle de medicamentos?

R: Acho essencial. Se bem feito, pode melhorar muito a adesão ao tratamento. Precisaria ter notificações confiáveis e um histórico que o paciente possa mostrar na consulta. Eu já vi alguns pacientes usando aplicativos similares, mas muitos reclamam que são complicados ou que as notificações não funcionam direito. Um aplicativo simples e confiável seria muito útil.

Q3: Que funcionalidades você considera mais importantes em um aplicativo desse tipo?

R: Lembretes automáticos são fundamentais, histórico detalhado, e se possível, exportar um relatório que eu possa ver na consulta. Também seria interessante poder cadastrar múltiplos medicamentos de uma vez, porque muitos pacientes tomam vários remédios. E claro, tem que ser muito fácil de usar, porque o público-alvo são pessoas que podem não ter muita familiaridade com tecnologia.

Q4: Há alguma funcionalidade que você considera crítica para o sucesso do aplicativo?

R: A confiabilidade das notificações é fundamental. Se o paciente não receber o lembrete, o aplicativo perde o propósito principal. Também é importante que seja muito simples de usar,

especialmente para idosos. Muitos pacientes idosos têm dificuldade com interfaces complexas. Se for muito complicado, eles desistem de usar.

Q5: Que tipo de informações você gostaria de ver em um relatório exportado?

R: Nome do medicamento, dosagem, horários prescritos, horários em que foi realmente tomado, e se houve atrasos ou esquecimentos. Isso me ajuda a avaliar a adesão ao tratamento. Seria interessante também ter uma visão geral de quantas vezes o paciente tomou corretamente versus quantas vezes esqueceu ou atrasou. Isso me dá uma ideia melhor de como está o tratamento.

Q6: Existe algum problema comum que você observa em pacientes que não seguem corretamente o tratamento?

R: O principal problema é esquecer de tomar, especialmente quando há múltiplos medicamentos em horários diferentes. Alguns pacientes também confundem os horários ou tomam doses erradas. Já tive casos de pacientes que tomaram o remédio duas vezes porque esqueceram que já tinham tomado. Outro problema comum é quando o paciente viaja ou muda a rotina e esquece de levar os medicamentos.

Q7: Como você acha que um aplicativo pode ajudar nesses casos?

R: As notificações são a chave. Se o celular tocar no horário certo, o paciente lembra. E o histórico ajuda tanto o paciente quanto eu a acompanhar se está seguindo corretamente o tratamento. Quando o paciente vê o histórico, ele mesmo percebe quando está falhando e pode se comprometer mais. Para mim, é muito útil porque na consulta eu posso ver exatamente o que aconteceu e ajustar o tratamento se necessário.

Q8: Há alguma preocupação com privacidade ou segurança dos dados?

R: Sim, dados de saúde são sensíveis. É importante que os dados fiquem apenas no celular do paciente, sem enviar para servidores externos. O paciente deve ter controle total sobre suas informações. Muitos pacientes têm receio de compartilhar informações de saúde, então a privacidade é crucial. Se houver qualquer dúvida sobre segurança, eles não vão usar.

Q9: Você já teve experiência com pacientes usando aplicativos de controle de medicamentos?

R: Sim, alguns pacientes já me mostraram aplicativos que usam. A maioria funciona, mas alguns são muito complicados ou têm problemas com notificações. O que mais vejo é paciente reclamando que o aplicativo não notifica direito ou que é difícil de configurar. Por isso acho importante focar na simplicidade e confiabilidade.

Q10: Que conselho você daria para quem está desenvolvendo esse tipo de aplicativo?

R: Testem com idosos desde o início. Se um idoso consegue usar facilmente, qualquer pessoa consegue. Foquem na simplicidade e na confiabilidade das notificações. E não esqueçam de pensar no relatório para o médico - isso é muito valioso para nós. Também seria interessante

ter uma forma de o cuidador acompanhar, para casos de pacientes que precisam de ajuda.

Observações do Entrevistador: O Dr. João demonstrou grande interesse no projeto e forneceu insights valiosos. Ficou claro que a simplicidade e confiabilidade são fatores críticos para o sucesso do aplicativo. A preocupação com privacidade também foi enfatizada várias vezes durante a conversa.

8.2 Apêndice B - Análise do Questionário

Total de respondentes: 52

Período: 01/12/2025 a 08/12/2025

Principais Resultados:

1. **Importância de Notificações:** 89% consideram muito importante
2. **Necessidade de Histórico:** 76% precisam de histórico detalhado
3. **Exportação de Relatórios:** 68% desejam exportar para médicos
4. **Simplicidade da Interface:** 82% priorizam interface simples
5. **Cadastro em Lote:** 54% gostariam dessa funcionalidade

Análise Detalhada dos Resultados:

Tabela 1 - Importância das Funcionalidades:

Funcionalidade	Muito Importante	Importante	Pouco Importante	Não Importante
Notificações	46 (89%)	5 (9%)	1 (2%)	0 (0%)
Histórico	40 (76%)	10 (19%)	2 (4%)	0 (0%)
Exportar PDF	35 (68%)	12 (23%)	4 (8%)	1 (2%)
Interface Simples	43 (82%)	7 (14%)	2 (4%)	0 (0%)
Cadastro em Lote	28 (54%)	16 (31%)	7 (13%)	1 (2%)

Observações:

- A funcionalidade de notificações foi considerada essencial pela maioria dos respondentes
- A necessidade de interface simples indica que o público-alvo prioriza usabilidade sobre funcionalidades complexas
- O cadastro em lote é desejável, mas não crítico para a maioria

8.3 Apêndice C - Relatório de Observação

Local: Farmácia Comunitária do Centro de Goiânia

Endereço: Rua 7, Centro, Goiânia-GO

Data: 03/12/2025

Horário: 14h às 17h

Observador: Victor Pereira (Estudante de Sistemas de Informação - IFG)

Perfil dos Observados:

- Total de pacientes observados: 15
- Faixa etária: 45 a 78 anos (média de 62 anos)
- 60% do gênero feminino, 40% masculino
- Todos possuíam smartphone e sabiam utilizá-lo

Comportamentos Identificados:

- 9 pacientes (60%) utilizavam anotações em papel para lembrar medicamentos
- 6 pacientes (40%) relataram esquecimento frequente de medicamentos
- 4 pacientes (27%) tinham dificuldade em lembrar os horários corretos
- 3 pacientes (20%) relataram tomar medicamentos no horário errado ocasionalmente
- 2 pacientes (13%) disseram que já esqueceram de tomar medicamentos importantes

Interações com Farmacêuticos:

- Farmacêuticos frequentemente precisavam relembrar horários e dosagens
- Houve pedidos de esclarecimento sobre como organizar múltiplos medicamentos
- Alguns pacientes pediram orientação sobre interações medicamentosas

Reações ao Conceito do Aplicativo:

- Todos os 15 pacientes demonstraram interesse ao saber da existência de um aplicativo de controle
- 12 pacientes (80%) disseram que usariam o aplicativo se fosse fácil de usar
- 10 pacientes (67%) mencionaram que o principal benefício seria não esquecer de tomar

- 8 pacientes (53%) valorizaram a possibilidade de mostrar histórico ao médico

Insights Importantes:

- O uso de papel ainda é comum, indicando necessidade de digitalização
- A confiança em anotações físicas pode ser uma barreira inicial à adoção
- A facilidade de uso é crucial, especialmente considerando a faixa etária dos usuários
- A integração com consultas médicas (através de relatórios) foi valorizada

Conclusões da Observação:

A observação confirmou a necessidade real de uma solução digital para controle de medicamentos. Os pacientes demonstraram abertura para tecnologia, desde que a interface seja simples e intuitiva. O aspecto de lembrete automático foi identificado como o diferencial mais valorizado.

Documento elaborado por: João Marcos, Sophia Lannah, Victor Gabriel

Data de conclusão: 03/02/2026